

ESCUELA TECNOLÓGICA INSTITUTO TÉCNICO CENTRAL

Tecnología en Desarrollo de Software
Electiva Técnica II - Ciencia de Datos

Optimización de rutas de última milla

Caso LogiExpress

Aplicación sobre el dataset Olist (Kaggle)



Integrante 1: Jhon Alejandro Correal Martinez

Integrante 2: Rafael Andres Guzman Rodriguez

Docente: Elias Buitrago

Semestre: Séptimo

Fecha: 8 de junio de 2026

Repositorio GitHub: <https://github.com/ACJUGGLER645/oli>

Resumen

Este informe documenta el desarrollo del caso LogiExpress: una empresa ficticia que debe entregar 30 pedidos en la ciudad de Sao Paulo desde un unico depposito, respetando una capacidad maxima por vehiculo. Se trabajo con el dataset publico de Olist (Kaggle), se calcularon las distancias entre puntos usando la formula de Haversine, se demostro experimentalmente que la solucion exacta no es viable para 30 pedidos y se construyeron las rutas con dos heurísticas sencillas: vecino mas cercano y 2-opt. El resultado final fueron **4 rutas factibles** con una distancia total de **161.60 km**, una mejora de 6.92 km frente al ordenamiento inicial.

Índice

1. Que problema estamos resolviendo	2
2. El dataset: de donde salen los datos	2
3. Preparacion de los datos	2
4. Como medir distancias entre dos puntos del mapa	3
5. Por que no podemos probar todas las rutas posibles	4
6. La regla del peso por vehiculo	5
7. Como armamos las rutas	6
7.1. Paso 1. Barrido angular	6
7.2. Paso 2. Vecino mas cercano (NN)	6
7.3. Paso 3. Mejora local con 2-opt	6
8. Resultados	7
9. Limitaciones	7
10. Conclusiones	9

1. Que problema estamos resolviendo

LogiExpress es una empresa ficticia de reparto urbano. Cada manana recibe varios pedidos desde un unico centro de distribucion (deposito) y debe entregarlos a clientes ubicados en distintas zonas de la ciudad. Cada pedido tiene una ubicacion (un punto en el mapa) y un peso. Cada vehiculo solo puede cargar hasta cierto peso maximo.

La empresa necesita dos cosas al mismo tiempo:

- **Que los pedidos quepan** en los vehiculos: ninguno debe exceder su capacidad.
- **Que se recorra la menor distancia posible**, para gastar menos combustible, tiempo y emisiones.

Este escenario es una version simplificada de un problema clasico de investigacion de operaciones llamado *Problema de Ruteo de Vehiculos con Capacidad*, conocido por sus siglas en ingles como CVRP (*Capacitated Vehicle Routing Problem*).

En palabras simples

Imagina que tu eres el repartidor. Te dan 30 paquetes para entregar, todos tienen una direccion y un peso, y tu camioneta solo puede cargar hasta 15 kilos por viaje. Tienes que decidir: cuantos viajes hacer, que paquetes meter juntos en cada viaje y en que orden visitarlos para manejar lo menos posible. Eso es exactamente el CVRP.

2. El dataset: de donde salen los datos

Nombre: Brazilian E-Commerce Public Dataset by Olist.

Fuente: Kaggle

Enlace: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

Olist es una marketplace de Brasil (algo parecido a Mercado Libre o Amazon). Publicaron en Kaggle un archivo abierto con casi 100 000 pedidos reales hechos entre 2016 y 2018. Los datos estan anonimizados: no aparecen nombres ni direcciones exactas, solo prefijos postales.

El dataset viene en **cinco archivos** que hay que combinar para poder armar un solo pedido con todo lo que necesitamos:

- `olist_orders_dataset.csv` - id del pedido y su estado.
- `olist_customers_dataset.csv` - cliente y prefijo postal.
- `olist_order_items_dataset.csv` - productos del pedido.
- `olist_products_dataset.csv` - peso de cada producto.
- `olist_geolocation_dataset.csv` - latitud/longitud por prefijo postal.

3. Preparacion de los datos

Los archivos originales no se pueden usar directamente. Hay que hacer varios pasos de limpieza:

1. **Unir las tablas:** se cruzan `orders` con `customers`, luego con `items`, productos y geolocalizacion. Asi cada fila queda con *id del pedido*, *coordenadas* y *peso* en un solo lugar.
2. **Resumir la geolocalizacion:** cada prefijo postal tiene muchas coordenadas distintas en el dataset (porque cubre varias cuadras). Tomamos la *mediana* de latitud y longitud por prefijo para representar el centro de esa zona.
3. **Calcular el peso del pedido:** sumamos el peso (en gramos) de los productos que lo componen y lo pasamos a kilogramos.
4. **Filtrar:** solo pedidos con estado `delivered` (entregados), ciudad Sao Paulo, peso mayor que cero, coordenadas dentro de Brasil.
5. **Seleccionar 30 pedidos:** se tomo una muestra aleatoria de pedidos con peso entre el percentil 50 y 90 (es decir, ni los mas livianos ni los mas pesados).

Tabla	Filas	Columnas
0	ordenes	99441
1	clientes	99441
2	items	112650
3	productos	32951
4	geolocalizacion	1000163

order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at	order_delivered_carrier_date	order_delivered_customer_date	order_estimated_delivery_date
0 e48151cbd54678b7cc491362d5a77	9e432aeb251297304e76186b10a28d	delivered	2017-10-02 10:56:33	2017-10-02 11:07:15	2017-10-04 19:55:00	2017-10-10 21:26:13	2017-10-18 00:00:00
1 53c2b2c8bc7dca0b741e2150273451	b0830f4747a6c6d20da0b8c802d7ef	delivered	2018-07-24 20:41:37	2018-07-26 03:24:27	2018-07-26 14:31:00	2018-08-07 15:27:45	2018-08-13 00:00:00
2 47770eb9100c290ca4946e9d07ac85d	41ca2a54c0b038f3443c3d31a357089	delivered	2018-08-08 08:38:49	2018-08-08 08:55:23	2018-08-08 13:30:00	2018-08-17 18:06:29	2018-09-04 00:00:00

customer_id	customer_unique_id	customer_zip_code_prefix	customer_city	customer_state
0 06b899e2ba1a1fbc8172c00ba8bc7	861ef4711a542a4b63843c6dd7f6bb0	14409	franca	SP
1 1895e83d33796b2d6eb18a428ac77	290c77bc52967ac935093aa66c333dc3	9790	sao bernardo do campo	SP
2 4e7b3a002865e6ebd8712d40374d03	060a732b5b29e6181a18229c790b2b5e	1151	sao paulo	SP

order_id	order_item_id	product_id	seller_id	shipping_limit_date	price	freight_value
0 000102426e8c5af6d1ba2d792b16214	1	4244733a06e7ecb4970a8e2683c13a61	48436dade18ac8b2bce089ec2a041202	2017-09-19 09:45:35	58.9	13.29
1 000187720320c57190d7a144bd43	1	e5f2d52b802189ee658865ca93d83a8f	dd7ddc04a1b6c2b0f14352b383efed236	2017-05-03 11:05:13	238.9	19.93
2 000229c388224ef0ca0657da4f703e	1	c777385d18b72bb7abbef9d44f50fd	5d51032edd3d242ad84c38acab88f23d	2018-01-18 14:48:30	199.0	17.87

product_id	product_category_name	product_name_lenght	product_description_lenght	product_photos_qty	product_weight_g	product_length_cm	product_height_cm	product_width_cm
0 1e9b8ef04dc8f4541ed26657ea517a5	perfumaria	40.0	287.0	1.0	225.0	16.0	10.0	14.0
1 3aa071139cb1067cae9e5dea641aaa2f	artes	44.0	276.0	1.0	1000.0	30.0	18.0	20.0
2 96bd78ac8810374ed1b6e5e291975717f	esporte_lazer	46.0	250.0	1.0	154.0	9.0	18.0	15.0

geolocation_zip_code_prefix	geolocation_lat	geolocation_lng	geolocation_city	geolocation_state
0 1037	-23.545621	-46.639292	sao paulo	SP
1 1046	-23.546081	-46.644820	sao paulo	SP
2 1046	-23.546129	-46.642951	sao paulo	SP

Figura 1: Dataset cargado en el notebook.

Que es un percentil

Si ordenas todos los pedidos del mas liviano al mas pesado, el percentil 50 es el peso del pedido que queda en la mitad. El percentil 90 es el que esta despues del 90 % de los pedidos. Tomar el rango 50–90 quiere decir: descarto los pedidos demasiado livianos (que no llenarian el vehiculo) y los demasiado pesados (que copan un vehiculo con uno o dos pedidos).

```

... Using Colab cache for faster access to the 'brazilian-e-commerce' dataset.
Descarga con kagglehub revisada en: /kaggle/input/brazilian-e-commerce
Usando dataset demo: False
Pedidos cargados: 30
Depósito definido: [-23.5675533902995, -46.64456321120669]

```

	Pedido_ID	Latitud	Longitud	Peso_kg	Ciudad	Estado	Cantidad_items
0	4888d985060eb4760da7373673343df2	-23.537368	-46.659642	1.60	sao paulo	SP	1
1	42e9b8d9802096afe88a27ff32fb838a	-23.665189	-46.646753	2.55	sao paulo	SP	1
2	a2a04ead650a00874aec73c0d989298b	-23.520018	-46.719062	1.00	sao paulo	SP	1
3	0c6715b2f888887f2ab9c7d6e099d38b	-23.523154	-46.445408	0.70	sao paulo	SP	1
4	4f6b1944e5dab6270cb4010b00edb3bc	-23.569763	-46.656772	0.70	sao paulo	SP	1
5	da8831dfbb89ea6b128840224899b348	-23.564658	-46.596266	0.98	sao paulo	SP	1
6	4ac657f0949f8f58a10b48c192df0b4e	-23.594000	-46.751503	1.01	sao paulo	SP	1
7	18827f4a6a534ed3ae539fe6f17a1b1	-23.606953	-46.523468	2.08	sao paulo	SP	1
8	6d0d1b394c3e9a042ef3fba7cb0968f5	-23.624000	-46.675273	1.98	sao paulo	SP	1
9	d61c0e8066e7a5a2e6071533b9efdd95	-23.701208	-46.634834	1.08	sao paulo	SP	2
10	0d21189c5494d8288ee358b3b24f85b4	-23.585995	-46.734721	1.15	sao paulo	SP	1

Figura 2: Tabla final con los 30 pedidos seleccionados y sus columnas: id, latitud, longitud, peso, ciudad, estado.

4. Como medir distancias entre dos puntos del mapa

Para construir rutas necesitamos saber que tan lejos esta cada pedido de los demas. Una idea ingenua seria usar la regla de Pitagoras sobre las coordenadas (latitud y longitud) como si fueran

un plano. Eso esta mal: la Tierra es *curva*, no plana, y un grado de longitud no mide lo mismo cerca del ecuador que cerca de los polos.

Lo correcto es usar la **formula de Haversine**, una formula con mas de 200 anos de antigüedad que aproxima la distancia sobre la superficie de una esfera (como si la Tierra fuera una pelota perfecta):

$$d = 2R \arcsin \left(\sqrt{\sin^2 \left(\frac{\varphi_2 - \varphi_1}{2} \right) + \cos \varphi_1 \cos \varphi_2 \sin^2 \left(\frac{\lambda_2 - \lambda_1}{2} \right)} \right) \quad (1)$$

Que significa cada simbolo

- d : la distancia entre los dos puntos, en kilometros. Es lo que queremos calcular.
- R : el radio de la Tierra. Usamos $R \approx 6371$ km.
- φ_1 y φ_2 : la *latitud* de cada punto (que tan al norte o al sur estan), expresada en radianes.
- λ_1 y λ_2 : la *longitud* de cada punto (que tan al este o al oeste estan), en radianes.
- $\sin$, $\cos$, $\arcsin$: funciones de trigonometria de secundaria.

La formula resuelve un triangulo sobre la esfera y devuelve la distancia mas corta que se podria recorrer sobre la superficie.

Con esta formula calculamos la distancia entre cada par de puntos: 30 pedidos mas el deposito dan 31 puntos, asi que la *matriz de distancias* es una tabla de 31×31 casillas. Se calcula una sola vez al principio y los algoritmos posteriores solo la consultan.

Limitacion. Haversine da la distancia en *línea recta* sobre la curvatura terrestre. No tiene en cuenta calles, sentidos viales, semaforos ni trafico. En la realidad la distancia recorrida sera mayor.

Nodo	0	1	2	3	4	5	6	7	8	9	...	21	22	23	24	25	26	27	28	29	30
0	0.000000	3.691594	10.858911	9.252620	20.893043	1.268373	4.932985	11.288008	13.094823	7.013388	...	2.070734	11.336726	4.600225	16.059056	13.766974	4.922043	0.177931	7.374937	6.306208	11.425328
1	3.691594	0.000000	14.273578	6.357644	21.898055	3.613950	7.137235	11.283290	15.889549	9.763807	...	3.015255	11.440765	8.059771	21.065903	11.069003	2.030629	3.520324	4.312829	6.600224	9.773940
2	10.858911	14.273578	0.000000	17.744436	25.891783	10.659926	12.305222	13.286618	14.129927	5.423660	...	11.384241	19.554525	6.261634	7.861923	24.498985	14.634979	11.033945	18.233239	12.516121	21.574567
3	9.252620	6.357644	17.744436	0.000000	27.902734	8.421010	13.465927	8.866073	22.155622	12.393624	...	7.412243	17.191906	12.328682	23.019015	13.416801	4.450593	9.129730	7.905852	14.933547	14.108380
4	20.893043	21.898055	25.891783	27.902734	0.000000	22.160316	16.055854	32.178561	12.252467	25.972271	...	22.796624	10.786739	22.623758	33.376837	19.212815	23.921374	20.880241	20.423848	14.900956	15.379908

Figura 3: Primeras filas de la matriz de distancias Haversine de tamaño 31×31 generada por el notebook.

5. Por que no podemos probar todas las rutas posibles

La forma “perfecta” de resolver el problema seria *probar todas las rutas posibles* y quedarse con la mas corta. Pero la cantidad de combinaciones crece monstruosamente rapido con el numero de pedidos.

Si tenemos n pedidos, la cantidad de ordenes distintos en que podemos visitarlos es $n!$ (se lee “n factorial”).

Que es n factorial

$n!$ significa multiplicar $1 \times 2 \times 3 \times \dots \times n$. Por ejemplo:

- $3! = 1 \times 2 \times 3 = 6$
- $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$
- $8! = 40320$
- $30! \approx 2,65 \times 10^{32}$ (un 265 seguido de 30 ceros)

Aun si un computador pudiera revisar mil millones de rutas por segundo, para 30 pedidos terminaria *mucho despues* de que el sol se apague. Por eso esta tecnica solo sirve para problemas muy pequenos.

En el notebook se hizo el experimento con n entre 3 y 8 pedidos:

Cuadro 1: Tiempo medido al probar todas las rutas posibles (fuerza bruta).

n	Rutas a probar	Tiempo (s)	Distancia (km)
3	6	0,0003	58,13
4	24	0,0012	65,01
5	120	0,0063	65,80
6	720	0,0502	68,00
7	5 040	0,3036	68,07
8	40 320	2,7162	68,08

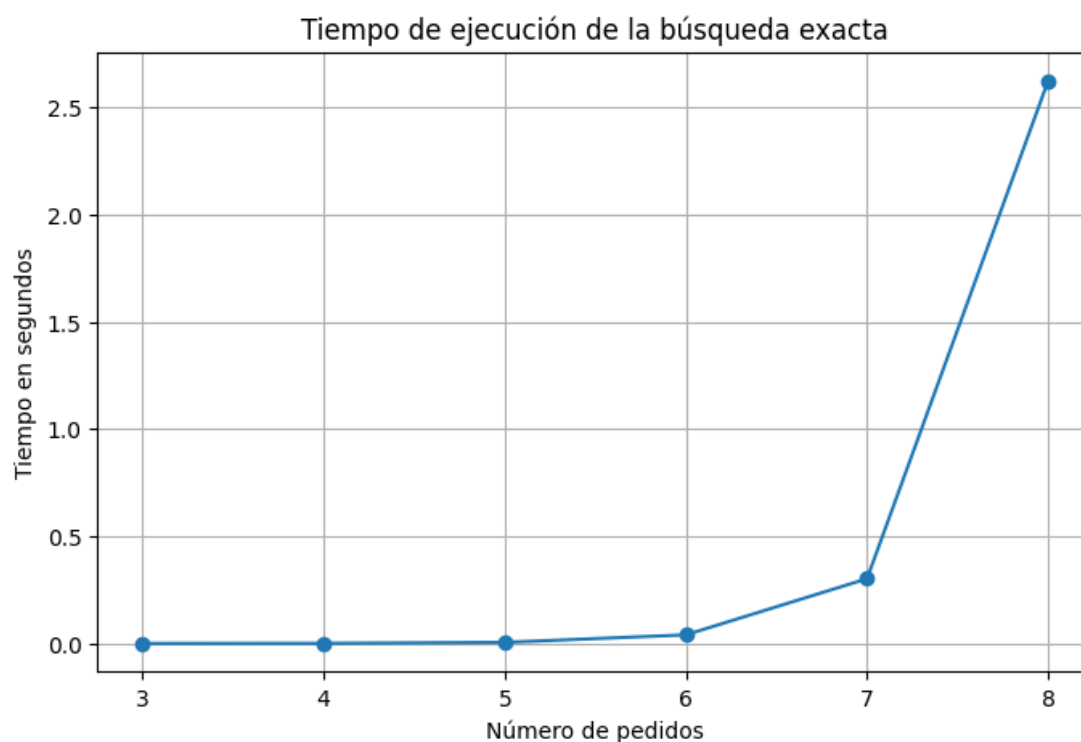


Figura 4: Grafica del tiempo de ejecución contra el número de pedidos. La curva crece muy rápido.

Conclusion: para 30 pedidos no podemos buscar la ruta perfecta. Hay que conformarse con una ruta *suficientemente buena* usando **heurísticas**.

Que es una heurística

Es una *receta* para conseguir una solución buena rápido, sin garantizar que sea la mejor. Es como cuando organizas tu maleta: no calculas la mejor configuración posible de todos tus cuadernos, solo los acomodas razonablemente y listo. En este informe usamos dos heurísticas: *vecino mas cercano* (NN) y *2-opt*.

6. La regla del peso por vehículo

Cada vehículo solo puede cargar hasta C kilogramos. Para cada ruta r formada por los pedidos que ese vehículo visita, la suma de los pesos no puede pasarse del límite:

$$\sum_{i \in r} \text{Peso}_i \leq C \quad (2)$$

Que significa cada simbolo

- r : una ruta, es decir, el grupo de pedidos que asignamos a un vehiculo.
- $i \in r$: cada pedido i que pertenece a esa ruta.
- Peso_i : el peso (en kilogramos) del pedido i .
- $\sum_{i \in r}$: el simbolo $\sum$ (sigma) significa *sumar todo*. Aqui sumamos los pesos de todos los pedidos de la ruta.
- C : la capacidad maxima del vehiculo, en kilogramos.

La formula entonces se lee: “la suma de los pesos de los pedidos de la ruta tiene que ser menor o igual a la capacidad del vehiculo”.

Decision tomada. La guia sugiere $C = 100$ kg. Pero los productos de Olist son pequenos (libros, ropa, accesorios) y con esa capacidad los 30 pedidos cabian en un solo vehiculo, por lo que no podiamos mostrar como funciona la restriccion. Cambiamos a $C = 15$ kg, que es coherente con una flota de motos o bicicletas electricas para reparto urbano de productos pequenos.

7. Como armamos las rutas

7.1. Paso 1. Barrido angular

Imaginamos al deposito en el centro y dibujamos una linea imaginaria hacia cada pedido. Esa linea tiene un *angulo*. Ordenamos los pedidos en sentido horario (como las manecillas del reloj) y vamos agrupandolos uno tras otro hasta que el siguiente no cabria en el vehiculo; ahi cerramos esa ruta y empezamos otra.

Esto se llama *sweep heuristic* (heuristica de barrido) y tiende a poner pedidos vecinos en el mismo vehiculo.

7.2. Paso 2. Vecino mas cercano (NN)

Dentro de cada ruta hay que decidir en que orden visitar los pedidos. La regla es muy simple: *desde donde estoy, voy al pedido no visitado mas cercano*. Se repite hasta haber visitado todos. Es rapido y casi siempre da una ruta razonable, aunque no la mejor.

7.3. Paso 3. Mejora local con 2-opt

2-opt es una idea sencilla: tomamos la ruta y *volteamos un tramo de ella* para ver si queda mas corta. Si si, nos quedamos con la nueva; si no, probamos otro tramo. Cuando ya ningun tramo se puede mejorar, paramos.

Por que se llama 2-opt

Porque en cada paso *cambia 2 conexiones* de la ruta: rompe dos “vias” que estaban conectadas y las reconecta de otra forma. Visualmente equivale a *deshacer cruces*: si la ruta dibuja una X, casi siempre se puede mejorar enderezando esa X.

Solucion optima vs. heuristica. La solucion *optima* es literalmente la mejor posible. Una *heuristica* entrega una buena pero sin garantia. 2-opt encuentra *minimos locales*: la ruta es mejor que sus parientes cercanos, pero puede existir una todavia mejor lejos que el algoritmo no encuentre.

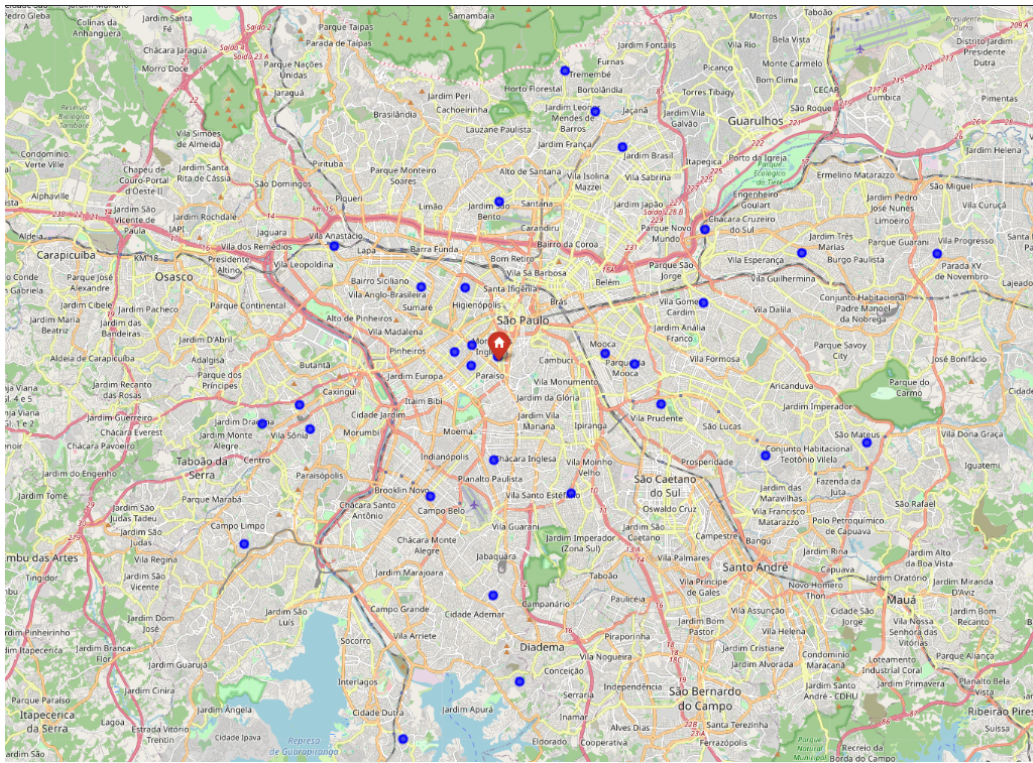


Figura 5: Mapa de los 30 pedidos seleccionados, con el deposito marcado en rojo.

8. Resultados

Cuadro 2: Resumen de las rutas generadas (capacidad = 15 kg).

Vehiculo	Pedidos	Peso (kg)	NN (km)	2-opt (km)	Mejora (km)
1	11	14,48	64,55	59,58	4,98
2	9	13,01	55,14	55,14	0,00
3	8	14,64	44,47	42,53	1,94
4	2	3,44	4,35	4,35	0,00
Total	30	45,57	168,51	161,60	6,92

Lectura de los resultados.

- Se generaron 4 vehiculos. Tres de ellos cargan cerca del limite (14 kg de los 15 kg posibles); un cuarto vehiculo lleva los 2 pedidos sobrantes.
- 2-opt logro mejorar las dos rutas con mas pedidos (vehiculos 1 y 3). Las rutas cortas (vehiculos 2 y 4) ya estaban en su minimo local y no admitieron mejora.
- La mejora total fue de 6,92 km, equivalente a un **4,1 %** menos respecto al ordenamiento inicial. Es una mejora modesta pero ganada en milisegundos de calculo.

9. Limitaciones

- Haversine entrega distancia en linea recta sobre la esfera. La distancia real por calles es mayor (semaforos, sentidos, rodeos).
- No se modelaron horarios de entrega, prioridades, tiempos de descarga ni jornada del conductor.
- Asumimos que todos los vehiculos son iguales. En la realidad suele haber flota mixta.

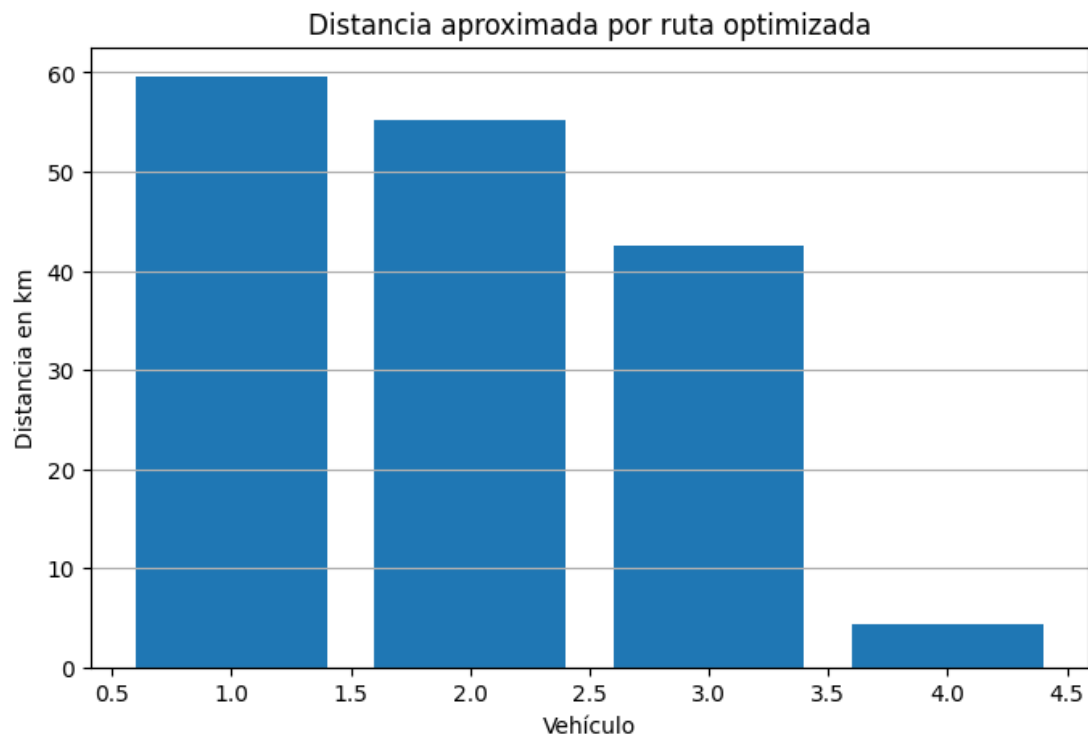


Figura 6: Distancia por vehiculo despues de aplicar 2-opt.

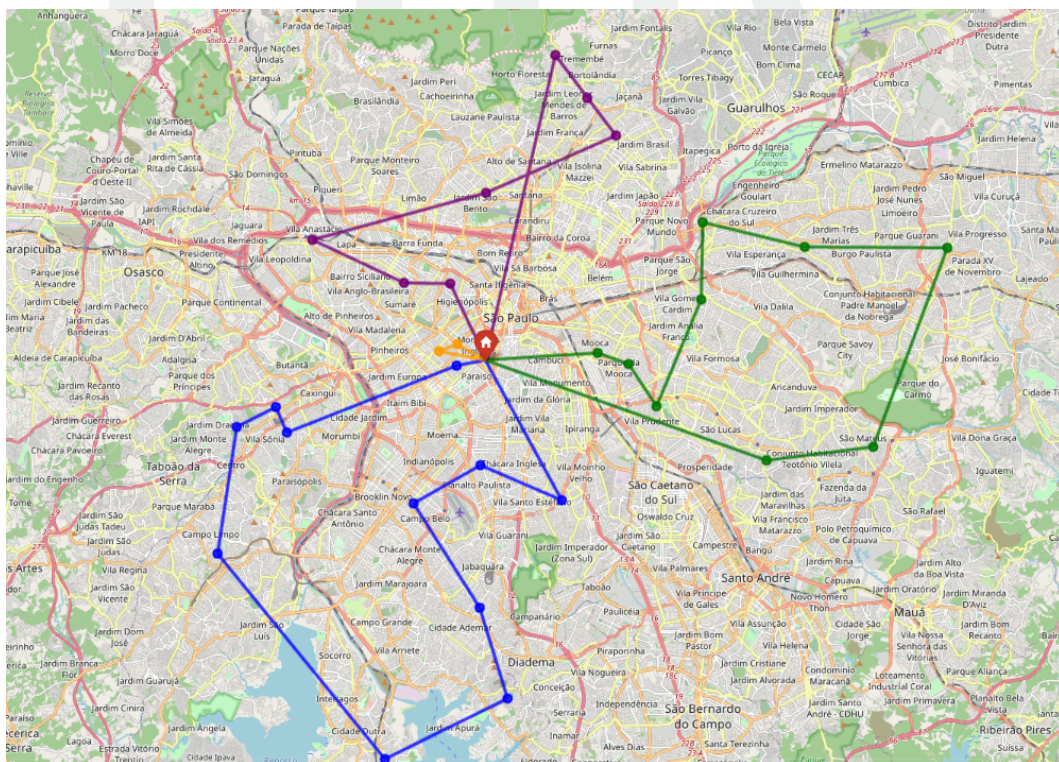


Figura 7: Mapa final con las cuatro rutas, una de cada color, y el deposito en rojo.

- El deposito es ficticio: se ubico en el centro geometrico de los pedidos.
- La geolocalizacion es por prefijo postal, no por direccion exacta del cliente.
- 2-opt encuentra un buen local, no garantiza el mejor global.
- Los datos son de 2016–2018; el comportamiento real evoluciona.

10. Conclusiones

- Logramos llevar un dataset publico real de principio a fin: lo descargamos, limpiamos, integramos, calculamos distancias y armamos rutas factibles.
- Confirmamos con un experimento que probar todas las rutas posibles para 30 pedidos es imposible en la practica. Por eso las heurísticas son necesarias en logistica real.
- La combinacion *barrido angular + agrupar por capacidad + vecino mas cercano + 2-opt* entrega rutas razonables en segundos y respeta la restriccion de carga.
- Decisiones como bajar la capacidad a 15 kg y filtrar el rango 50–90 del peso fueron necesarias para que el ejercicio muestre la dinamica del problema real.
- Para acercarlo a una operacion real habria que sumar: ventanas horarias, distancias por calles, flota mixta y prioridades de cliente.

Anexo. Archivos producidos

- `Optimizacion_Rutasv3.ipynb` - notebook ejecutado.
- `pedidos_logiexpress_limpio.csv` - los 30 pedidos.
- `matriz_distancias_haversine.csv` - matriz 31×31.
- `scrolllytelling.html` - version web del informe.

ETITC